



FAKULTAS
ILMU
KOMPUTER

Web Design

Using HTML5 and CSS3

Tim Dosen PBP

Static vs Dynamic Website

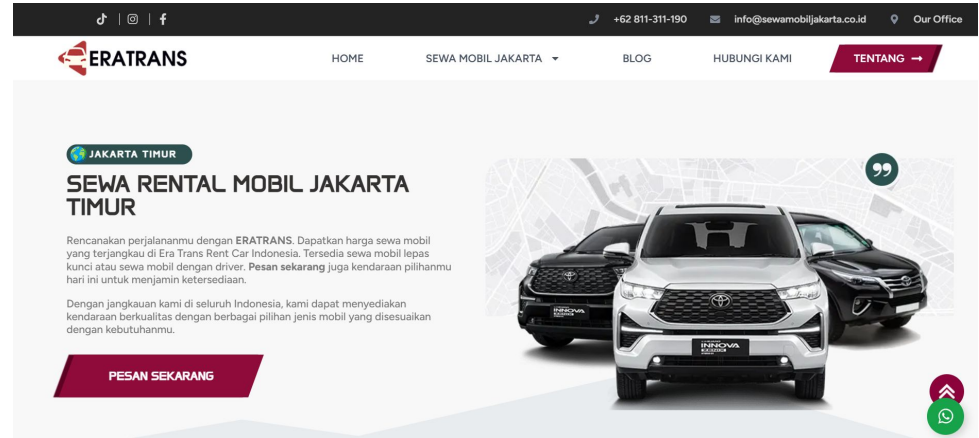
- **Static website** is designed to deliver pre-rendered HTML files directly to the user's browser, offering a fixed presentation where visitors cannot directly interact with the underlying data. Because the content is pre-built, every user sees the exact same information upon visiting. Any updates or changes to the displayed content require a developer to manually edit and re-deploy the source code.
- **Dynamic website** is designed to instantly adapt its content based on user interaction and real-time data. Dynamic sites rely on a combination of server-side programming and databases to process every individual request. This mechanism allows the system to generate unique, relevant, and personalized experiences tailored specifically to each visitor.

Introduction to Web Design

Which one do you trust more for renting a car?



A

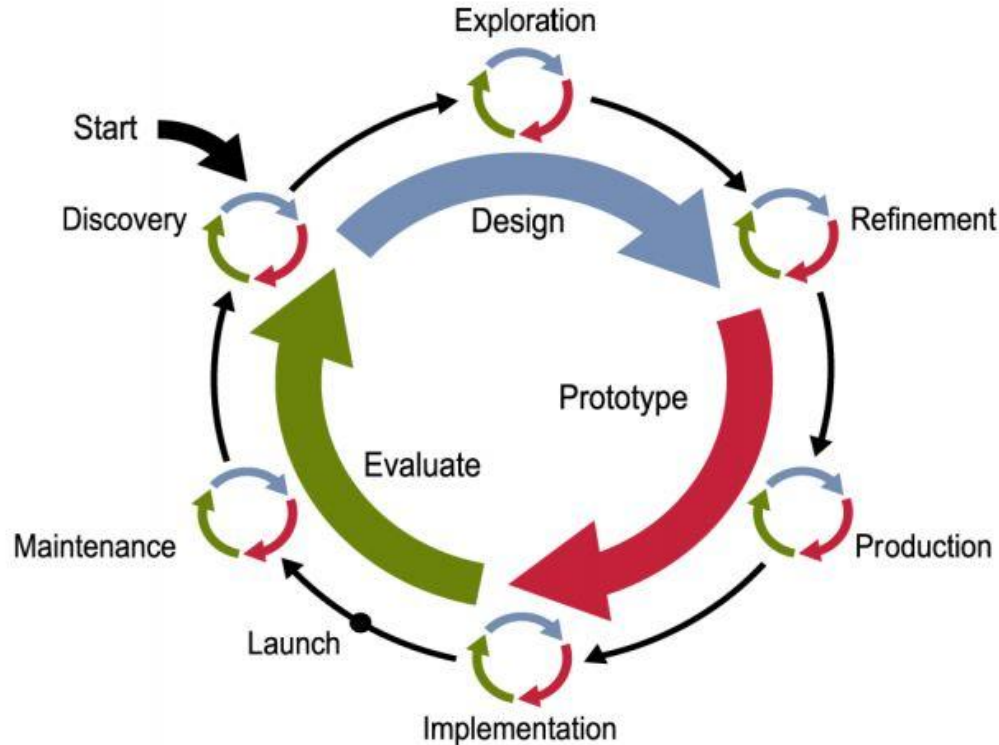


B

Why does web design matter?

- A positive first impression
- Enhance credibility and trust
- Improved SEO
- Improve User engagement

Iteration Process



Discovery



**Design
Exploration**



**Design
Refinement**



Production

Setting Goals and Web Objectives

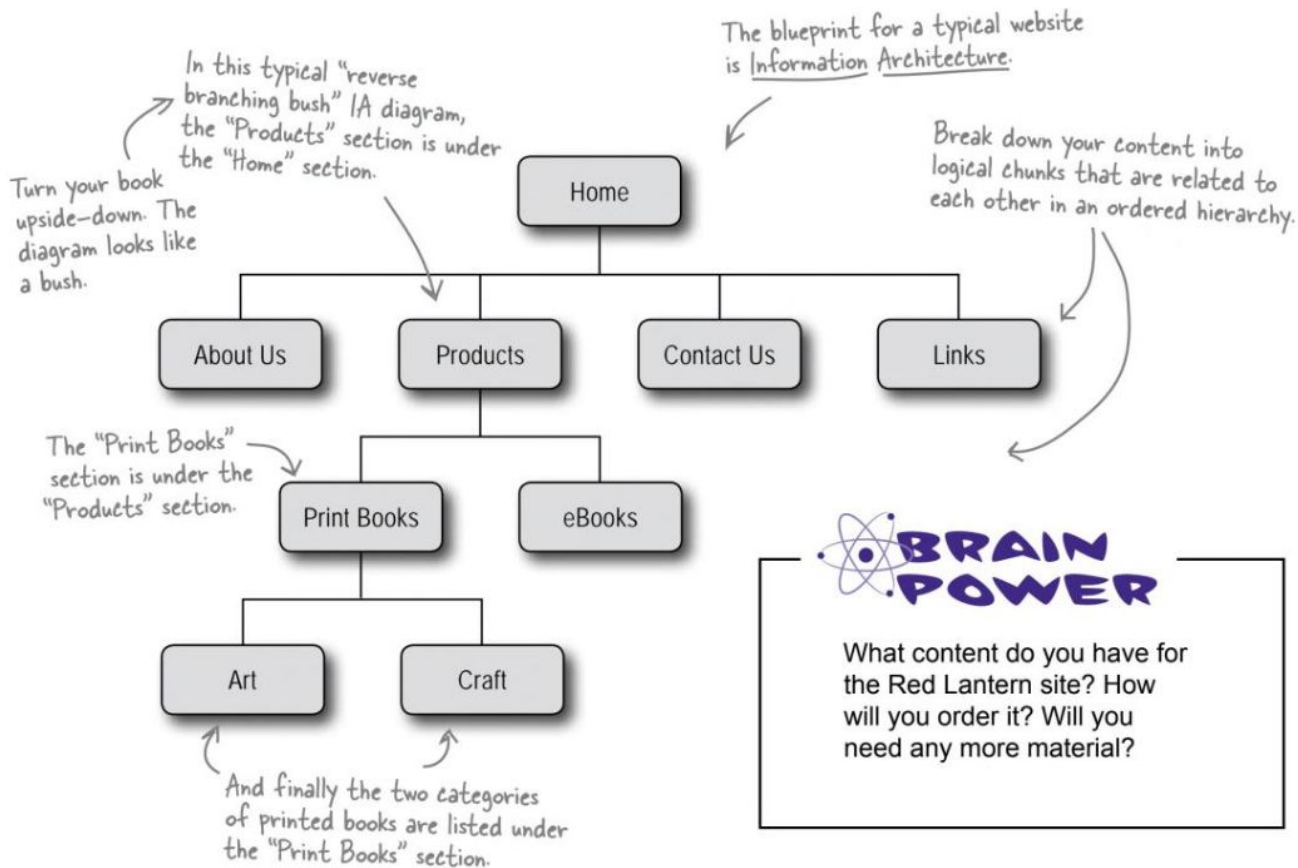
#1 Discovery

Value Proposition (VP) Questions

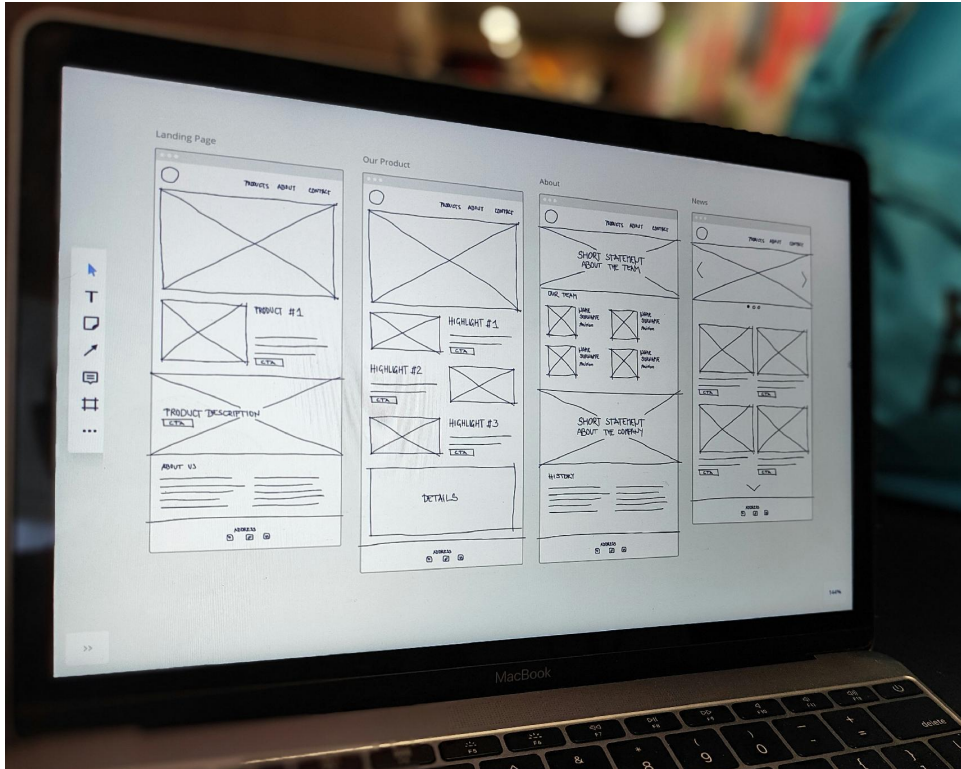
1. What are the purposes/visions of this website?
2. What problems will this website solve?
3. Whom will this website help?
4. How will this website help them?

Design Exploration

Information Architecture



Sketching and Wireframing



- Outlining structure or content of the website without visual design
- Wireframe focus on functionality, user flow and content

Designing Interfaces

#2 Design Exploration & Refinement

The 8 Golden Rules of Interface Design

1. Strive for consistency
2. Seek universal usability
3. Offer informative feedback
4. Design dialogs to yield closure
5. Prevent errors
6. Permit easy reversal of actions
7. Keep users in control
8. Reduce short-term memory load

Source: <https://www.cs.umd.edu/users/ben/goldenrules.html>

Shneiderman's 8 Golden Rules of Interface Design

The principles	Questions to consider	Mark Complete
1. Strive for consistency	Is the style of this element maintained across your site/app? Is this content placed in the correct location according to the site hierarchy? Does this follow the conventions for your chosen platform? How can you make your designs more consistent?	☐
2. Enable frequent users to use shortcuts	Are there shortcuts available for your more experienced users? Who is this product designed for? Will there be a need to consider experienced users? How can you make it easier and quicker for experienced users?	☐
3. Offer informative feedback	Does the user know where they are at in the process? Does the user know what they have done after performing this action? How are you communicating this feedback to your user?	☐
4. Design dialogue to yield closure	Does the user have to do any guessing here? Is it clear and obvious enough for your intended audience? Are there any next steps for the user? How are you communicating the system status with the user?	☐
5. Offer simple error handling	Have you done everything imaginable to prevent this error from happening on your end? Is this error avoidable in the first place? If the user does make an error, how easy is it for them to fix it?	☐
6. Permit easy reversal of actions	How many steps does the user have to take to reverse their actions? Will the user quickly realize they need to reverse the action in the first place? How can you make your users detect the possibility of reversal?	☐
7. Support internal locus of control	Will the user feel in control at this specific touch point in your app? Will they be surprised in an unpleasant manner? Does the site feel easily navigable? Does the user feel safe and in control? How can you make the user feel more safe and in control??	☐
8. Reduce short-term memory load	Are there enough visual cues here for the user to find the functionality or item? Do they have to remember things to understand what's going on? How can you help the user recall?	☐

Source: <https://public-media.interaction-design.org/pdf/Shneiderman.s.Eight.Golden.Rules.Worksheet.pdf>

The Psychology of Color

RED

POSITIVE

Power
Passion
Energy
Fearlessness
Strength
Excitement

NEGATIVE

Anger
Danger
Warning
Defiance
Aggression
Pain

ORANGE

POSITIVE

Courage
Confidence
Warmth
Innovation
Friendliness
Energy

NEGATIVE

Deprivation
Frustration
Frivolity
Immaturity
Ignorance
Sluggishness

YELLOW

POSITIVE

Optimism
Warmth
Happiness
Creativity
Intellect
Extraversion

NEGATIVE

Irrationality
Fear
Caution
Anxiety
Frustration
Cowardice

GREEN

POSITIVE

Health
Hope
Freshness
Nature
Growth
Prosperity

NEGATIVE

Boredom
Stagnation
Envy
Blandness
Enervation
Sickness

TURQUOISE

POSITIVE

Communication
Clarity
Calmness
Inspiration
Self-expression
Healing

NEGATIVE

Boastfulness
Secrecy
Unreliability
Reticence
Fence-sitting
Aloofness

BLUE

POSITIVE

Trust
Loyalty
Dependability
Logic
Serenity
Security

NEGATIVE

Coldness
Aloofness
Emotionless
Unfriendliness
Uncaring
Unappetizing

PURPLE

POSITIVE

Wisdom
Luxury
Wealth
Spirituality
Imaginative
Sophistication

NEGATIVE

Introversion
Decadence
Suppression
Inferiority
Extravagance
Moodiness

MAGENTA

POSITIVE

Imaginative
Passion
Transformation
Creative
Innovation
Balance

NEGATIVE

Outrageousness
Nonconformity
Flippancy
Impulsiveness
Eccentricity
Ephemerality

BROWN

POSITIVE

Seriousness
Warmth
Earthiness
Reliability
Support
Authenticity

NEGATIVE

Humorlessness
Heaviness
Unsophisticated
Sadness
Dirtiness
Conservativeness

BLACK

POSITIVE

Sophistication
Security
Power
Elegance
Authority
Substance

NEGATIVE

Oppression
Coldness
Menace
Heaviness
Evil
Mourning

GRAY

POSITIVE

Timelessness
Neutrality
Reliability
Balance
Intelligence
Strength

NEGATIVE

Unconfident
Dampness
Depression
Hibernation
Lack of energy
Blandness

WHITE

POSITIVE

Cleanliness
Clarity
Purity
Simplicity
Sophistication
Freshness

NEGATIVE

Sterility
Coldness
Unfriendliness
Elitism
Isolation
Emptiness

Color Palette



 Chatbolt.ai Create your own ChatGPT chatbot trained with your data from any file or website. [Try It For Free](#)

Tools 

[Go Pro](#)

Sign in

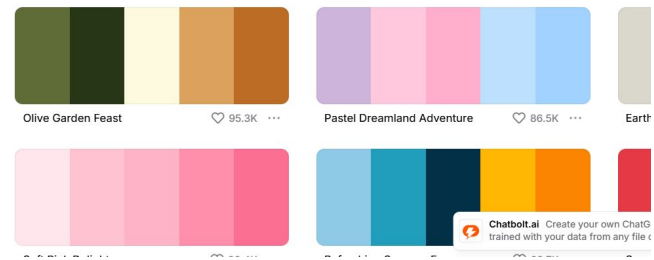
[Sign up](#)

 Search with colors, topics, styles or hex values...



Trending Color Palettes

Get inspired by thousands of beautiful color schemes and make something cool!



ColorKit

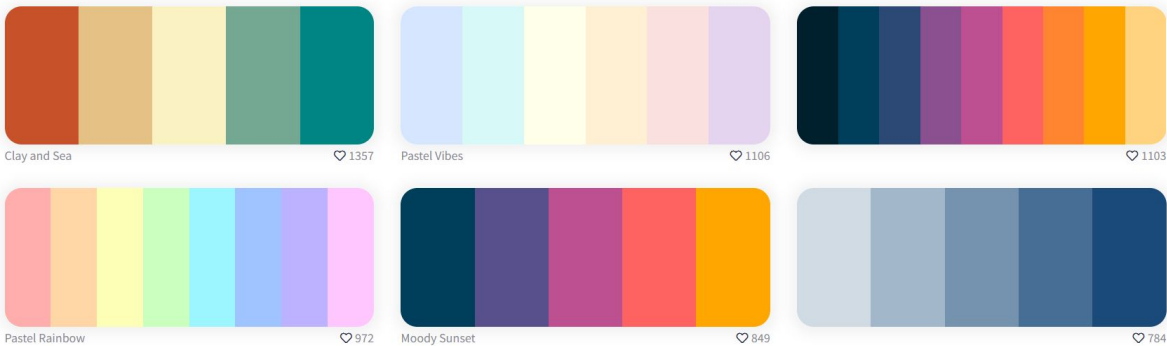
[Color Palette Generator](#) [Palettes](#) [Gradients](#) [Tools](#)  [Blog](#) [Sign Up](#) [Login](#)

Color Palette Inspiration

Top color palettes created with our [color palette generator](#)

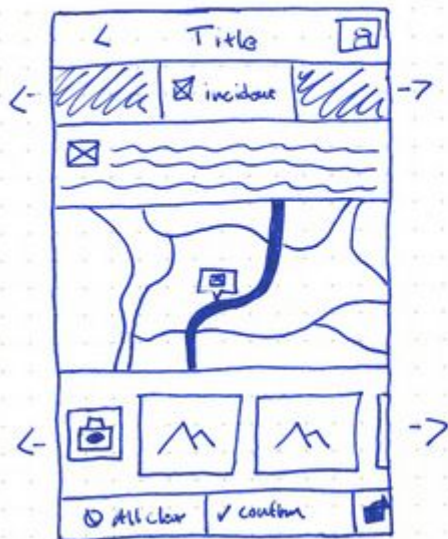
Search for colors, styles, and more

 Sort [+ Create Palette](#)



Prototyping

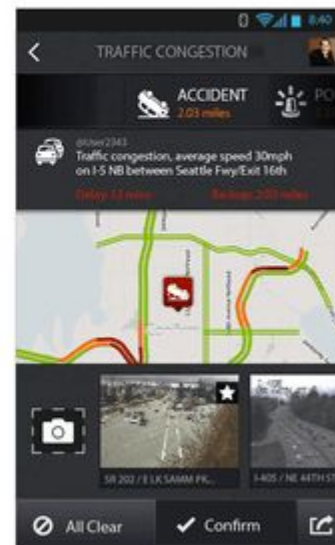
SKETCH



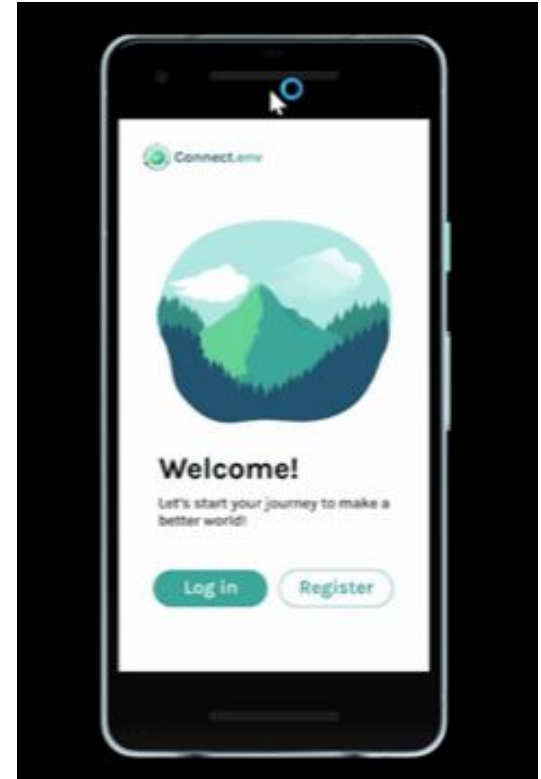
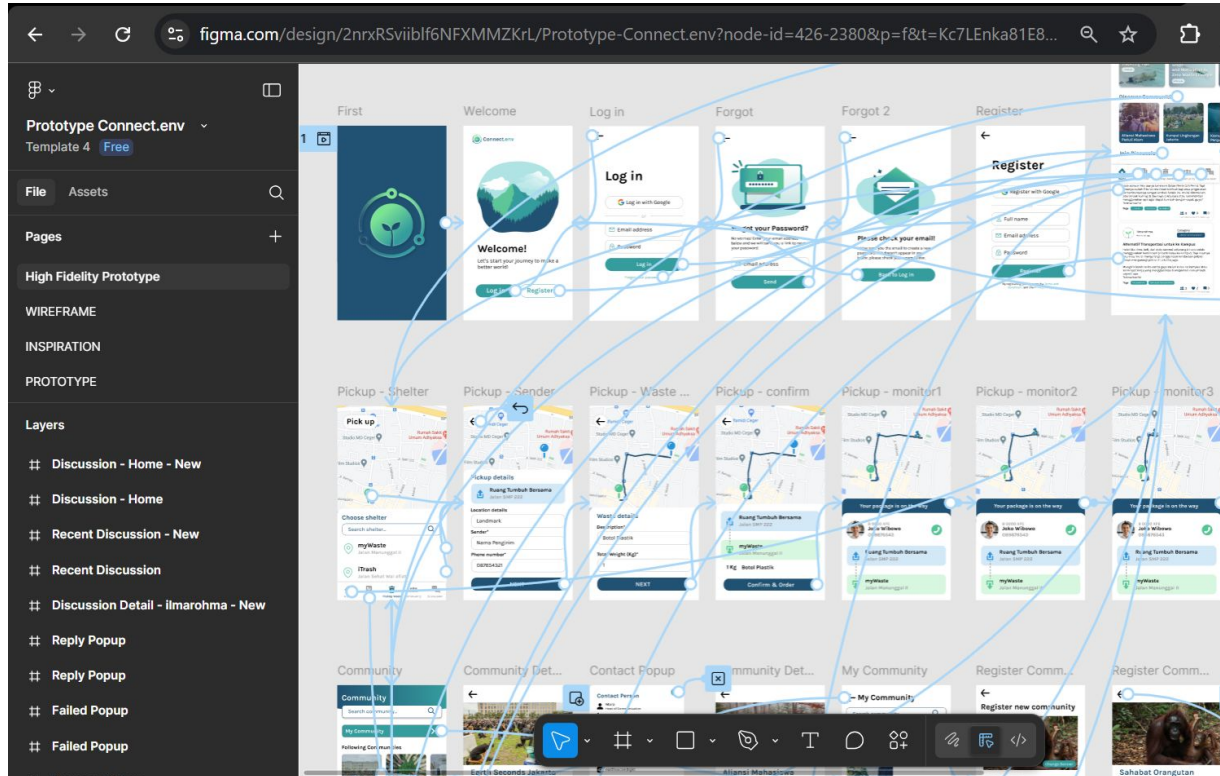
LOW-FI



HI-FI



Figma: Prototyping tools



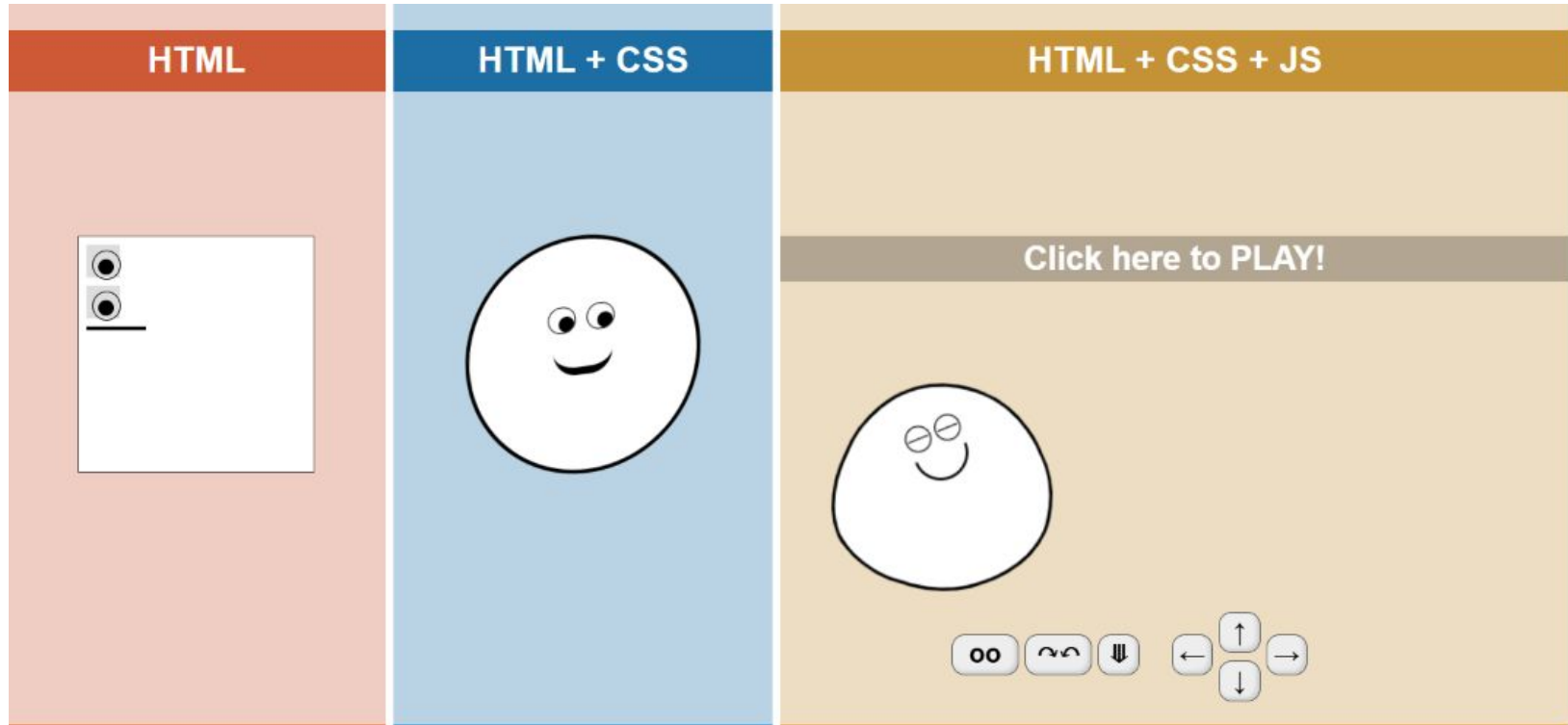


Design is not just what it
looks like and feels like.
Design is how it works.

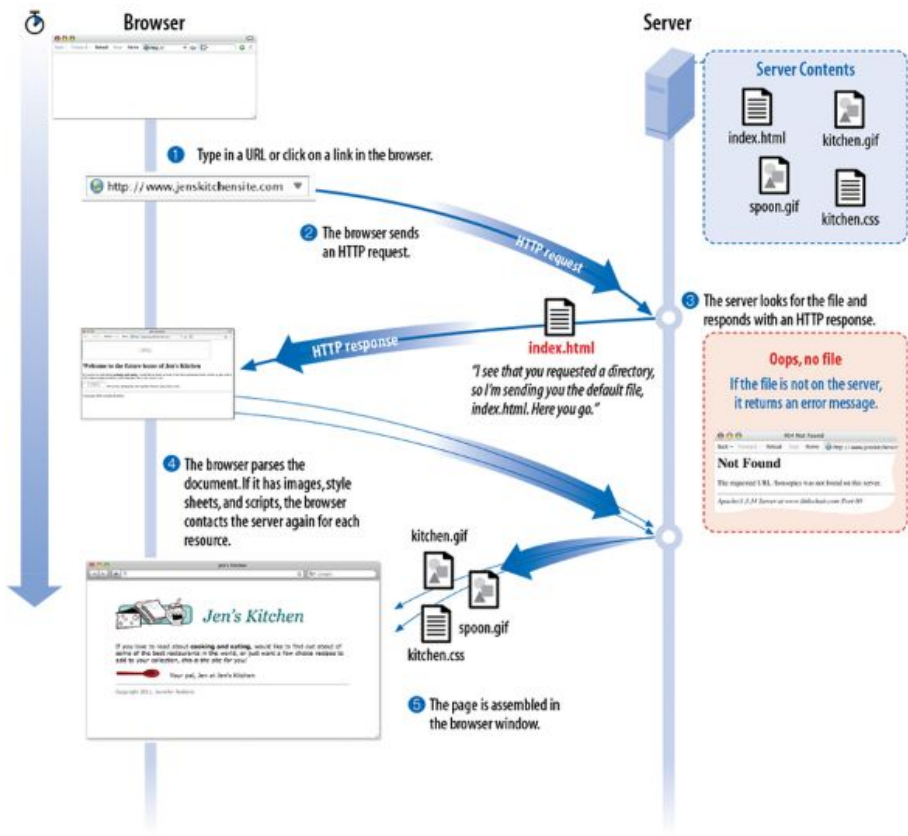
Steve Jobs

“ quote fancy

The Anatomy of a Web Page



How do they work together?



HTML5

Hypertext Markup Language

Evolution of HTML

1993	HTML 1.0 - Developed by Tim Berners Lee to link the document
1995	HTML 2.0 - Developed by Internet Engineering Task Force RFC to include stylized text and tables
1996	CSS1
1997	HTML 3.2 - Developed by W3C and included browser specific feature
1997	HTML 4.0 - A move back to normalizing the pages across platforms
1998	CSS2
1999	HTML 4.01 - Introduced different document types
2012	HTML 5 - Back to HTML plus multimedia and semantic tags

HTML and Browser

	MAC				WIN							
												
	FIREFOX	OPERA	CHROME	SAFARI	FIREFOX	OPERA	CHROME	SAFARI	IE			
	11	11.62	18	5.1	11	11.61	18	5.1	6	7	8	9
Canvas	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✓ 90%
Canvas Text	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✓ 89%
SVG	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✓ 88%
SVG Clipping Paths	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✓ 88%
SVG Inline	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✓ 54%
SMIL	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✓ 69%
WebGL	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✓ 61%
Audio	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✓ 88%
Video	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✓ 88%

First HTML Example

```
<> index.html X
<> index.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>PBP Gajil 25/26</title>
5    </head>
6    <body>
7
8      <h1>Pemrograman Berbasis Platform </h1>
9      <p>Pemrograman Berbasis Platform (PBP) merupakan mata ajar yang melatih dan
10         memberikan pengalaman pengembangan aplikasi berbasis platform baik dari
11         sisi server maupun di sisi client, memperkenalkan pengembangan platform
12         Web dan platform Mobile. Peserta perkuliahan juga dituntut untuk bisa
13         melakukan pemrograman dengan bahasa Python, Dart, Java, HTML, CSS, dan
14         JavaScript.
15      </p>
16
17    </body>
18  </html>
```



HTML Element

- Suatu HTML Element didefinisikan dengan start tag, content dan end tag

<code><tagname></code>	<code>Content here</code>	<code></tagname></code>
Start tag	Content	End Tag

``

Self Closing Tag

HTML Elements

Tag	Description
<code><!DOCTYPE></code>	Defines the document type
<code><html></code>	Defines an HTML document
<code><head></code>	Contains metadata/information for the document
<code><title></code>	Defines a title for the document
<code><body></code>	Defines the document's body
<code><h1> to <h6></code>	Defines HTML headings
<code><p></code>	Defines a paragraph
<code>
</code>	Inserts a single line break

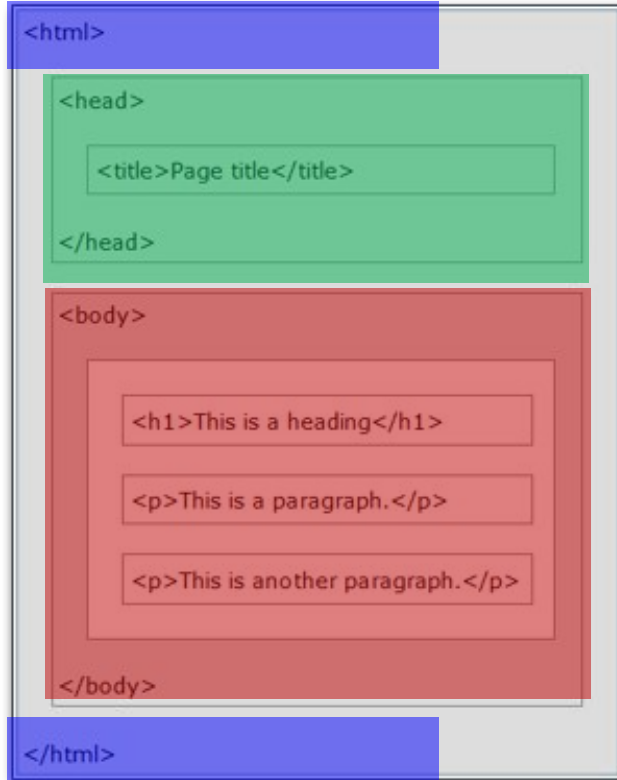
- For more elements : <https://www.w3schools.com/tags/default.asp>

HTML Attributes

- Provide **additional information** about elements
- Always specified in **the start tag**
- Usually come in name/value pairs like: **name="value"**
- Provide information to browser on how to handle the element
 - Styling-related attributes should be defined in CSS (more on this later!)

```
<section name="form">
  <h2>Send Message</h2>
  
  <form id="msg_form" method="POST" action="add_message">
    <input type="text" name="id_text" required/>
    <button type="submit" class="btn btn-default">Submit</button>
  </form>
</section>
```

HTML Basic Page Structure



- `<html>` is used to inform the browser that this is an HTML Document
- `<head>` is used to contain metadata such as document title, character set, styles, links, scripts; not all information in this section is displayed to the user
- `<body>` contains the content of the documents

HTML5

HTML5 is the latest specification of the HTML language, with objectives primarily include:

- **Encouraging semantic** (meaningful) markup
- **Separating design** from content
- Promoting **accessibility** and design **responsiveness**
- **Reducing the overlap** between HTML, CSS, and JavaScript
- Supporting **rich media** experiences while eliminating the need for plugins such as Flash or Java

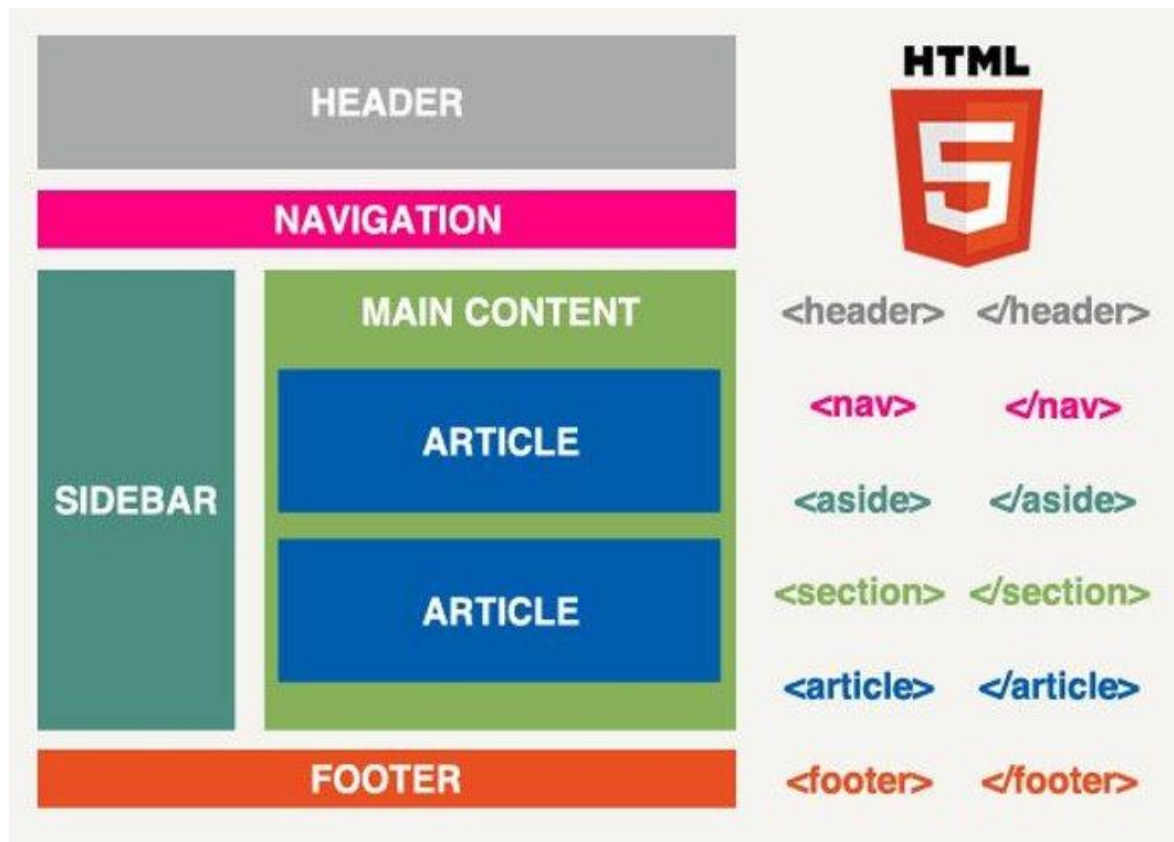
Read more:

<https://html.com/html5/#ixzz77q499mnd>

What's New?

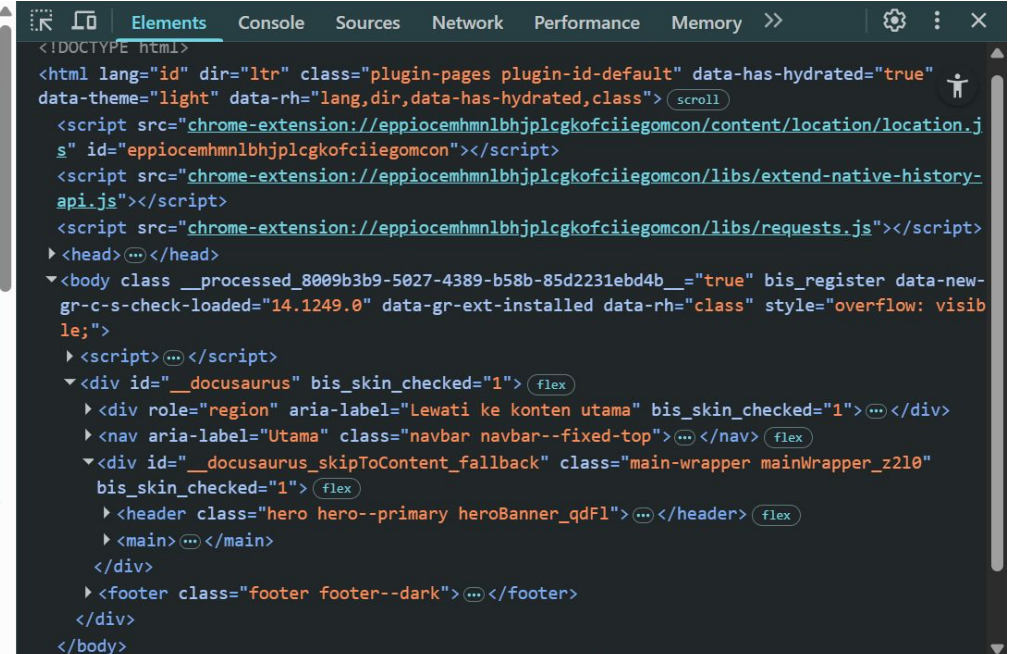
- semantic elements like `<nav>`, `<header>`, `<footer>`, `<article>`, `<section>`.
- attributes of form elements like `data-list`, `keygen`, `output`
- input types: `datetime`, `number`, `email`, `month`, `url`, `color`, etc
- input attributes: `required`, `placeholder`, `autofocus`
- graphic elements: `<svg>` and `<canvas>`.
- multimedia elements: `<audio>` and `<video>`.

HTML 5 Body Structure Elements



HTML in Real Web

- can you spot the HTML structure of a PBP web bellow?



Question: Is HTML a programming language?

- To be more specific, we can break down the question to:
 - Is HTML a **language**?
 - Hint: Recall the discussion about regular expression & context-free grammar in Automata & Theory of Language (ATL) course!
 - Is HTML a **programming language**?
 - Hint: Will be answered once your ATL course discusses about the Turing machine

HTML alone is boring, how do we design our web pages to be attractive?

CSS3

Cascading Style Sheets

CSS (Cascading Style Sheets)

- CSS is a language that **describes the style** of an HTML document.
- CSS describes how selected **HTML elements should be displayed**.
- CSS allows you to **specify the presentation of elements** on a web page (e.g., fonts, spacing, sizes, colors, positioning) **separately** from the document's structure and content (section headers, body text, links, etc.)

HTML + CSS

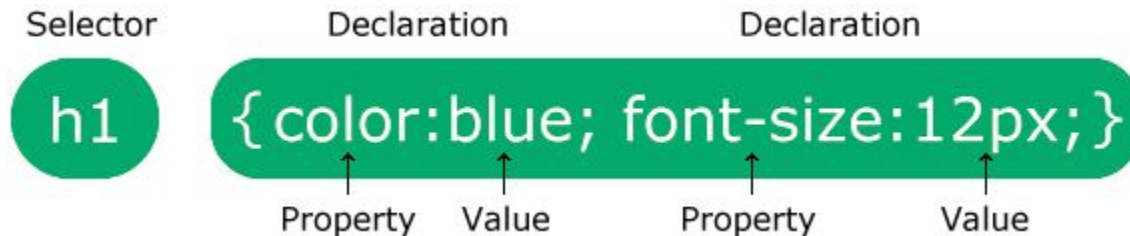
HTML + CSS



HTML



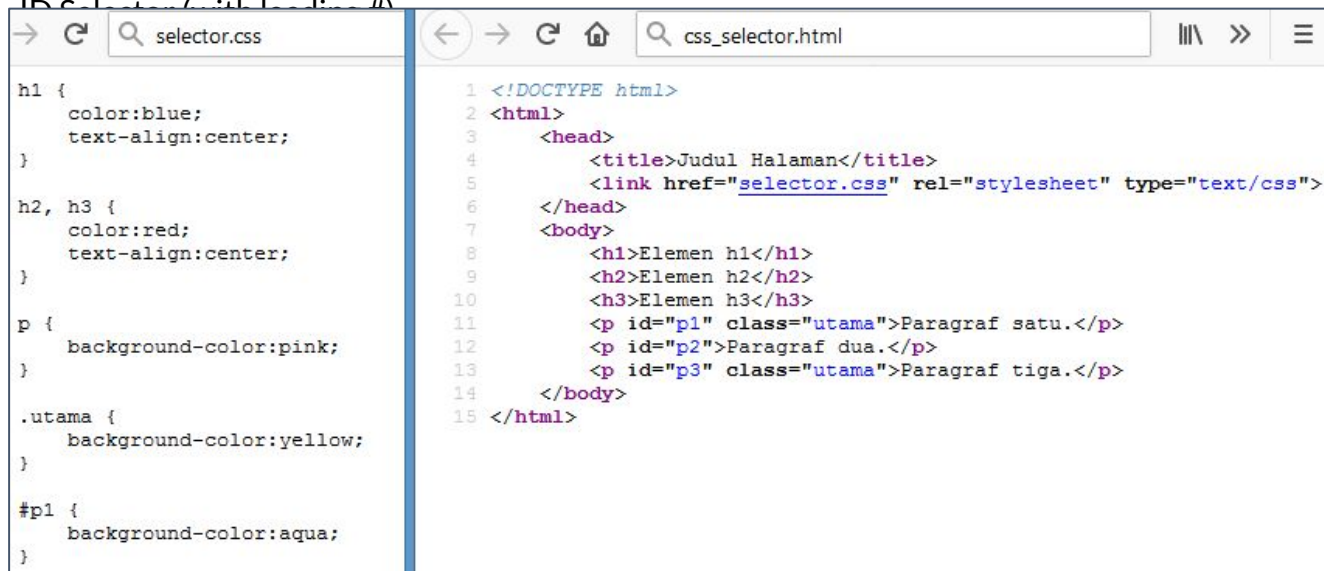
CSS Syntax



- The **selector** points to the HTML element you want to style.
- The **declaration** block contains one or more declarations separated by **semicolons**.
- Each declaration includes a CSS **property name** and a **value**, separated by a colon.

CSS Selector

- used to "find" (or select) the HTML elements you want to style.
- The HTML element can be selected by:
 - Element Selector (without leading # or .)
 - Class Selector (with leading .)
 - ID Selector (with leading #)

A screenshot of a code editor with two files open. The left file, 'selector.css', contains CSS rules for h1, h2/h3, p, and .utama. The right file, 'css_selector.html', contains an HTML document with a title, a link to the CSS file, and three paragraphs with IDs p1, p2, and p3, all using the 'utama' class.

```
selector.css
h1 {
  color:blue;
  text-align:center;
}
h2, h3 {
  color:red;
  text-align:center;
}
p {
  background-color:pink;
}
.utama {
  background-color:yellow;
}
#p1 {
  background-color:aqua;
}
```

```
css_selector.html
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Judul Halaman</title>
5     <link href="selector.css" rel="stylesheet" type="text/css">
6   </head>
7   <body>
8     <h1>Elemen h1</h1>
9     <h2>Elemen h2</h2>
10    <h3>Elemen h3</h3>
11    <p id="p1" class="utama">Paragraf satu.</p>
12    <p id="p2">Paragraf dua.</p>
13    <p id="p3" class="utama">Paragraf tiga.</p>
14  </body>
15 </html>
```

CSS Selector : Combinator

Combinator	Description	Example
Descendant combinator (space)	matches all elements that are descendants (children, grandchildren, etc.) of a specified element.	<code>div p {...}</code> Selects all p elements that are descendants of a div element
Child combinator (>)	selects all elements that are direct children of a specified element.	<code>div > p {...}</code> Selects all p elements that are children of a div element
Next sibling combinator (+)	used to select an element that is directly after a specific element.	<code>div + p {...}</code> Selects the first p element immediately after the div element (must have the same parent element)
Subsequent-sibling combinator (~)	selects all elements that are next siblings of a specified element.	<code>div ~ p {...}</code> Selects all p elements that are aligned and immediately after the div element

How to apply CSS

- Internal Style Sheet

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-color: linen;
}
h1 {
  color: maroon;
  margin-left: 40px;
}
</style>
</head>
<body>

<h1>Pemrograman Berbasis Data</h1>
<p>Mata Kuliah Pemrograman Berbasis Data Ganjil 2025/2026</p>

</body>
</html>
```

- Inline Style

```
<!DOCTYPE html>
<html>
<body>

<h1 style="color:blue;text-align:center;">
  Pemrograman Berbasis Data
</h1>

<p style="color:red;">
  Mata Kuliah Pemrograman Berbasis Data Ganjil 2025/2026
</p>

</body>
</html>
```

How to apply CSS (cont'd)

- External style sheet (Recommended for better maintenance)

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="mystyle.css">
</head>
<body>
<h1>Pemrograman Berbasis Data</h1>
<p>Mata Kuliah Pemrograman Berbasis Data Ganjil 2025/2026</p>
</body>
</html>
```

index.html

```
body {
  background-color: lightblue;
}

h1 {
  color: navy;
  margin-left: 20px;
}
```

mystyle.css

CSS3 Flexbox

- CSS3 is the latest standard in CSS and it is backward compatible
- Flexbox is a (new) layout mode in CSS3
 - https://www.w3schools.com/css/css3_flexbox.asp
 - <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

Try:

- https://www.w3schools.com/css/tryit.asp?filename=trycss3_column-count
- https://www.w3schools.com/css/tryit.asp?filename=trycss3_flexbox_wrap-reverse

Where is the “Cascading” part?

If there are two rules still in conflict at a given weight, the algorithm will continue to “cascade down” and check attribute priorities until it finds one that wins.

Priorities (highest order wins)

1. Origin & Importance
2. Selector Specificity
3. Order of Appearance
4. Initial & Inherited Properties (default values)

1. Origin & importance

- Origin

1. **Author:** CSS comes from front-end developer.
2. **User:** The user of browser can manually set the style for their browser, such as its font style and color.
3. **User-Agent:** The browser serves default style. Different browser provides different style

- The importance of a CSS declaration is determined by the **!important** syntax.

- + use **!important** as an escape hatch for when all else fails (such as when working with 3rd-party styles)
- can make your CSS more brittle

2. Selector Specificity (Selectors)

- CSS selectors can belong to one of the following order.
The highest order wins.
 1. Inline styles (anything inside a style tag)
 2. ID selectors
 3. Classes selector
 4. Element selector

Selector Specificity (Selectors)

```
...  
<h1 class="class1" style="color: red;" id="title1">HTML5 Example Page</h1>  
...
```

style.css

```
h1 {  
    color: blue;  
}  
#title1 {  
    color: aqua;  
}  
.class1 {  
    color: cadetblue;  
}
```

Apa warna tulisan "HTML5
Example Page"?

3. Source Order

If you've got 2 stylesheets linked in the head of your HTML document, the second stylesheet will override rules in the first stylesheet.

First CSS loaded

```
<html>
  <head>
    <link href="file.css"
          rel="stylesheet"
          type="text/css">

    <style type="text/css">
      table {
        color: red;
      }
    </style>
  </head>
</html>
```

Second CSS loaded. If there are same element selectors named "table", then the Second CSS will override the style.

4. Initial & inherited properties

- In the following example, the `<p>` tag will render with a monospace font & red text, since its parent node contains those styles.
- The `<div>` tag will render with yellow background, inherited from style from its parent (the style for `<body>` tag)
- The “inherit” property will automatically use the value from its parent. When the parent change its style, their child inherit all changed style.
- For non-inherited properties, each element has a set of initial values.

```
<html>
  <head>
    <style type=text/css>
      div {
        background-color: initial;
        color: inherit;
      }
      body {
        background-color: yellow;
      }
    </style>
  </head>
  <body>
    <div style="font-family: monospace; color: red;">
      <p>inheritance can be super useful!</p>
    </div>
  </body>
</html>
```

Beyond the CSS

Framework to Help You Design Better

Prettification: Using a CSS Framework

- Design is hard, and doubly so now that we have to deal with mobile, tablets, and so forth.
- That's why many programmers are turning to CSS frameworks to solve some of those problems for them.
- There are lots of frameworks out there, but one of the earliest and most popular is Twitter's Bootstrap.

Responsive Design using Bootstrap

- Framework for building responsive, mobile-first sites, with the BootstrapCDN and a template starter page.
- Bootstrap also provides several animated and interactive elements built by JavaScript.

Learning Source:

- <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

How to use Bootstrap?

1. Copy the Bootstrap source link to your HTML code or download Bootstrap and add the Bootstrap folder to your CSS Folder.
2. Find the Bootstrap style on the documentation.
3. Put the Bootstrap class to your code.

Example: 1. Copy the Bootstrap source link to your HTML code

Include Bootstrap's CSS and JS. Place the `<link>` tag in the `<head>` for our CSS, and the `<script>` tag for our JavaScript bundle (including Popper for positioning dropdowns, poppers, and tooltips) before the closing `</body>`. Learn more about our [CDN links](#).

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap demo</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/css/bootstrap.min.css" re
  </head>
  <body>
    <h1>Hello, world!</h1>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.2.0/dist/js/bootstrap.bundle.min.
  </body>
</html>
```

Example: 2. Find the Bootstrap style on the documentation

Buttons

[View on GitHub](#)

Use Bootstrap's custom button styles for actions in forms, dialogs, and more with support for multiple sizes, states, and more.

Examples

Bootstrap includes several predefined button styles, each serving its own semantic purpose, with a few extras thrown in for more control.



HTML



```
<button type="button" class="btn btn-primary">Primary</button>
<button type="button" class="btn btn-secondary">Secondary</button>
<button type="button" class="btn btn-success">Success</button>
```

Example: 3. Put the Bootstrap class to your code

```
<body>
  <button>
    Button (without Bootstrap)
  </button>
  <button type="button" class="btn btn-primary">
    Button (with Bootstrap)
  </button>
</body>
```

Button (without Bootstrap)

Button (with Bootstrap)

**Components**

- Accordion
- Alerts
- Badge
- Breadcrumb
- Buttons
- Button group
- Card
- Carousel**
- Close button
- Collapse
- Dropdowns
- List group
- Modal
- Navbar
- Navs & tabs
- Offcanvas
- Progress

Carousel

A slideshow component for cycling through elements—images or slides of text—like a carousel.

[View on GitHub](#)

How it works

The carousel is a slideshow for cycling through a series of content, built with CSS 3D transforms and a bit of JavaScript. It works with a series of images, text, or custom markup. It also includes support for previous/next controls and indicators.

In browsers where the [Page Visibility API](#) is supported, the carousel will avoid sliding when the webpage is not visible to the user (such as when the browser tab is inactive, the browser window is minimized, etc.).

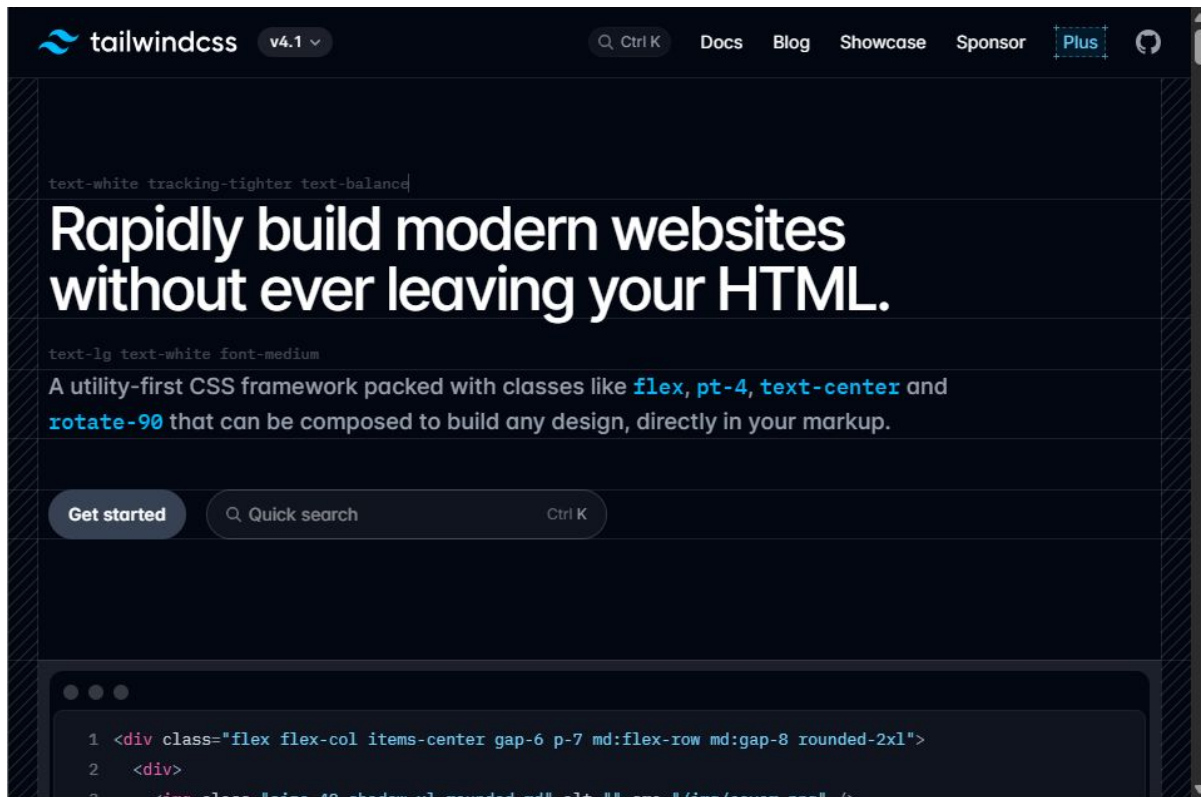
The animation effect of this component is dependent on the `prefers-reduced-motion` media query. See the [reduced motion section of our accessibility documentation](#).

Please be aware that nested carousels are not supported, and carousels are generally not compliant with accessibility

On this page

[How it works](#)[Example](#)[Slides only](#)[With controls](#)[With indicators](#)[With captions](#)[Crossfade](#)[Individual .carousel-item interval](#)[Disable touch swiping](#)[Dark variant](#)[Custom transition](#)[Sass](#)[Variables](#)[Usage](#)[Via data attributes](#)[Via JavaScript](#)[Options](#)[Methods](#)

Other Framework



Tailwind is
pretty popular
on industry
nowadays

References

- Django Documentation: <https://docs.djangoproject.com/en/5.0/>
- CSS3 Flexbox:
 - <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
 - https://www.w3schools.com/css/css3_flexbox.asp
 - https://www.w3schools.com/css/tryit.asp?filename=trycss3_column-count
 - https://www.w3schools.com/css/tryit.asp?filename=trycss3_flexbox_wrap-reverse
- Bootstrap: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- <https://html.com/html5/#ixzz77q499mnd>
- Watrall, E., J., Siarto. Head First Web Design.. O'Reilly Media, Incorporated, 2009.